Building in the Age of AI

Prompt pack from the workshop. Zoltán Fodor · zoltanfodor.dev

The idea in one line

AI didn't take the work, it took the waiting. What didn't speed up is your job: taste, knowing the customer, and making the call. **Cut the waiting, keep the craft.**

Start with the task you hate most

Don't start your AI transformation with a strategy deck. Start with the one task you hate most, because you know it inside out (so you can judge the output), it comes back every week (so it pays back), and you feel the relief on day one.

```
I want to hand a task I hate to AI. The task: [describe it in 2–3 sentences].
Interview me with 5 short questions, one at a time, about how I do it today, what
good output looks like, and what must never go wrong.
Then write: 1) the steps as a checklist, 2) the instructions for a Project / Gem that
does it with me, 3) what I still check myself every time.
```

Your first week with AI

Monday	Pick the task you hate most.
Tuesday	Make one Project / Gem with your context: product, customers, glossary, voice, templates.
Wednesday	Connect one tool through MCP (a "connector": Drive, Gmail, Notion, Slack…).
Thursday	Write down what you never cut: customer data, checking, the decision.
Friday	Let it run. After two weeks, keep it or drop it.

Brief it like a smart new hire

The wording matters far less than the context. Every good prompt has five parts:

Role	Context	Task	Format	Check
who should it be?	your docs, customers, rules	one clear job, in steps	table, list, one page	"no data" is an allowed answer; quote sources

Three levels

1 Chat: you ask, it answers. **2 Project:** with your own docs and rules (the biggest single jump). **3 Agent:** it gets a goal, plans, does, checks, and repeats until done. Aim for level 2 this week.

We cut

- ~~the waiting~~
- ~~the perfect first draft~~
- ~~the spec nobody reads~~

We never cut

- ✓ protecting customer data (business account, training off)
- ✓ checking every number, law and quote
- ✓ the decision: that's yours

From idea to launch: the prompts we used

1. Validate with real customer words

```
Role: a senior product researcher.
Context: [your idea in 1–2 sentences].
Task:
1. Search public sources for what real users say about the problem and the existing solutions: app store reviews, G2 / Capterra, Reddit and forums. Leave out people building or selling a tool.
2. Group the complaints into 5–7 themes.
3. For every theme: how many separate posts or reviews mention it (no percentages), 2 short verbatim quotes, each with its link, marked (opened) or (snippet).
4. The 3–5 existing solutions people mention, and what they like and hate about each.
5. Top 3 opportunities (frequency × pain).
Format: one table of themes, then at most 10 bullets.
Check: only use what you actually found. Every quote needs a link. If something cannot be found, write "no data". Do not invent numbers.
```

Then: "*List every quote behind the top theme with its link.*" Open a few yourself.

2. A simple business case

```
Write a simple business case for this idea: the problem and for whom, impact hypotheses, an assumptions table (assumption | pessimistic | base | optimistic | how we would validate it), the result for the 3 scenarios with the calculation shown, risks, what we would measure in a 2-week test, and a verdict: Go, Narrow or Stop. Clearly mark what is an assumption and what is a fact. Do not invent market data; write "to validate" instead.
```

3. Red team your own plan (the most valuable prompt)

```
Review this plan through four pairs of eyes: 1) a skeptical engineer, 2) the customer who says no and would never pay, 3) an investor, 4) a bank's risk and compliance officer (customer data, regulation, fraud). Each writes the 3 biggest gaps or risks. Then update the plan and list what changed.
```

Good design with AI

"Make it modern and clean" gets you the same generic AI look as everyone else. Instead: **real references → your rules → three variants side by side → you pick, it refines → check it on a real phone.**

```
[attach 2–3 screenshots of apps you admire]
Build a clickable mobile prototype (one HTML page) from this spec: the main list
screen with 6–10 realistic made-up records, a detail view, settings if needed, and an
empty state.
Use the screenshots as the visual reference for spacing, type sizes, density and
tone. Do not copy their brand.
Make three variants side by side (A, B, C) that differ in layout, not just colour.
```

Write your taste down once

```
From the chosen variant, write a short design-rules.md: colours (hex), font family
and sizes, spacing scale, corner radius for controls and for cards, what is banned
(for example gradients, heavy shadows), and a rule that every screen has an empty
state. At most 15 lines.
```

Attach that file to every design or code request. If you correct the same thing twice, add it to the file.

When a chat gets long: hand off, start fresh

Long chats get worse: early rules fade, things in the middle get lost. Don't argue for twenty more messages. Ask for a handoff, open a new chat, paste it in.

```
Write a handoff so a fresh chat can continue this work without reading this
conversation. Max 15 lines:
1. Goal, in one sentence.
2. Decisions made so far, and why (one line each).
3. What is done, what is not.
4. Open questions, and who decides them.
5. The exact next step.
6. Files, links or numbers the next chat must have.
Leave out everything we tried and dropped, unless the reason matters.
```

Automate the boring operations

```
Every weekday at 8:00:
1. Read new emails in the support inbox from the last 24 hours.
2. Group them by topic and write a 5-line summary for me.
3. Draft a reply to each one, using the tone and answers in my "support-rules" file.
Do NOT send anything.
4. Create a ticket for every bug report, quoting the customer's words.
5. Put anything urgent at the top.
```

It drafts, you approve. Anything that sends, pays or deletes waits for you.

After the workshop: five more prompts

Same shape as the live ones: role, context, task, format, check. Each one drafts; you read it, fix it and send it yourself. (Prompts 8-12 in the repo's workshop/prompts.md.)

After a sales call: the follow-up

```
Role: a founder-led sales assistant.
Context: my raw notes (or the transcript) from a call with [name, role, company].
[paste]
Task: draft the follow-up email I send today.
Format: one line thanking them for something specific they said; their problem in
their own words (one quote); what we agreed, at most 3 bullets; the next step with a
date and who does it. Under 150 words. Below the email, separately: anything they
asked that I did not answer.
Check: promise nothing that is not in my notes, and no feature that does not exist
today. If the notes have no agreed next step, say so instead of inventing one.
```

The monthly investor update

```
Role: my chief of staff.
Context: this month's numbers [paste], what went well, what went badly, what I need
help with. Last month's update: [paste, or "this is the first one"].
Task: draft this month's investor update in the short early-stage format: asks first,
then highlights, lowlights, milestones.
Format: asks at the top, at most 3, each concrete enough that one person could act on
it (a named kind of company, a role to hire, a specific intro); 1-2 key numbers vs
last month; 2-5 highlights; 2-5 lowlights; next month's milestones.
Check: the lowlights stay in, in plain words. Every number matches what I pasted,
with the calculation shown for any derived one. Flag any ask too vague to act on
("intros to customers").
```

The format is Elad Gil's: "Investor Update Emails", blog.eladgil.com.

Competitor teardown

```
Role: a skeptical market analyst.
Context: my product in one paragraph: [...]. The competitor: [name, website].
Task: a teardown from public sources only: their site, pricing page, docs, changelog,
app store and review sites.
Format: a table: who they sell to | what it does | price and plan structure | what
customers praise | what customers complain about, every cell with a link and the date
you saw it. Then 3 places we are clearly better, 3 where they are, and one question
to ask our next customer about them.
Check: praise and complaints are verbatim quotes, marked (opened) or (snippet).
Anything you cannot find is "not public", not a guess. No claims about their revenue
or customer count unless a source says it.
```

Meeting notes into decisions and owners

```
Role: a meticulous note-taker.
Context: the transcript or my rough notes from [meeting]. [paste]
Task: turn it into a record someone who was not there can act on.
Format: decisions made (one line each, who decided); actions (what, owner, due date);
open questions; anything discussed but left undecided.
Check: an action without an owner or a date is listed as "no owner" or "no date"; do
not assign one. A suggestion nobody agreed to is not a decision. Quote where the
exact wording matters.
```

Contract first read (not legal advice)

```
Role: a careful reader preparing me for a conversation with my lawyer. You are not my
lawyer and this is not legal advice.
Context: a contract I am asked to sign: [type, my side, the other side]. [paste or
attach]
Task: help me understand what I would be agreeing to and what to ask.
Format: 1) a 5-line plain-English summary; 2) a table: clause (section number and a
short quote) | what it means for me | unusual or one-sided? | question for my lawyer.
Cover at least term and termination, payment, liability caps, indemnities, IP
ownership, data protection, auto-renewal, exclusivity; 3) the 3 questions I should
ask first.
Check: every point quotes the clause with its section number. A clause you would
expect but cannot find is "not found". Do not tell me whether to sign.
```

Before you paste a contract, check your plan: business and team plans don't train on your chats by default; on a personal plan, turn model training off in Settings first. Don't rate the answer with a thumb: that sends the whole chat to the vendor. Anything that binds you to money, liability or IP still gets a real lawyer; this makes that meeting shorter, not unnecessary.

Where to start

Assistant	ChatGPT, Claude or Gemini
On your own docs	NotebookLM
Prototype	Figma Make, Lovable, Claude
Full project	Lovable, Claude Code
Start a real app from	a template, not a blank page: Vercel templates (AI chatbot, next-forge), v0
Meeting notes	Gemini in Meet, Granola
Agent	Claude Cowork, ChatGPT Work

Know where it stops: AI is faster and better on first drafts, summaries, prototypes and ideas, and confidently wrong on your customers, fresh rules, local law and numbers without a source. Always ask for the source.

Slides, every prompt, skills, guides and 300+ links: **github.com/ritmillio/building-in-the-age-of-ai**